Module lifecycle, packaging and billing boundaries: audit execution prompt

Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated review function. Record a finding, a verified strength or a reasoned not-applicable decision for every relevant lens. Recommendations must follow repository evidence. Do not claim security, accounting, legal, tax, accessibility, privacy or release certification from a checklist or passing unit tests.

Project: Stoquify. **Workspace:** E:/ohada saas/Focused projects/stoquify. **Module/workflow:** MOD — Module lifecycle, packaging and billing boundaries. **Mission:** audit this domain from data and source events through authorization, services/actions, asynchronous handoffs, queries, UI, exports and operational recovery; then produce an executable correction plan.

1. Objective and boundaries

Determine what is correct, incomplete, inconsistent, fragile, inefficient, unsafe, intentionally deferred or unverified. Preserve implemented controls and identify the most material operational, data, financial, privacy and reputational consequences. This is an audit and plan: write reports/diagnostics only; do not implement production changes, alter records, apply migrations, seed databases or deploy.

Complete these domain workflows:

1. Evaluate catalog, durable grant and permission authority on reads/writes.
2. Stop new work while retaining mandatory settlement, corrections and historical access.
3. Reconcile dependencies and commercial/provider changes without direct billing authority over business records.

2. Review responsibilities

Use architecture; backend/distributed integration; database/migrations; security/IAM/privacy/fraud; frontend/design system; UX/accessibility/EN-FR; product/process; domain financial controls; quality/release; SRE/performance; and packaging/operations perspectives. Activate qualified statutory review only for actual jurisdiction-dependent claims. AI is a connected-boundary review unless this is the agent domain; do not propose an AI feature merely to fill a lens. Proposed accountable owner: Platform engineering and commercial product operations.

3. Evidence-first discovery

Read applicable AGENTS.md and the docs-report-publisher skill. Record commit, branch, dirty source hashes and time before review. Preserve existing work and older evidence. Start with services/modules/module-entitlement.service.ts and actions/modules/module-composition.actions.ts, prisma/schema.prisma, relevant committed migration SQL, types/queryKeys.ts, package.json and the graphify-out graphs. Expand callers/consumers and find active, legacy, duplicate, bypass and dormant paths.

Use the module review as a navigation lead, not proof that findings remain current. Hash and recheck every reused result. Candidate source families: `services/modules`, `actions/modules`, `lib/modules`, `components/modules`, `app/[locale]/(dashboard)/dashboard/settings/modules`. Graph nodes and communities in the review are historical navigation; current source wins.

4. Architecture and source ownership

Inventory capabilities, routes, APIs, actions, services, workers, adapters, tables, constraints, hooks, screens, reports and permission keys. For every workflow map business event → authoritative record → fresh authorization → service/transaction → event/outbox → dependent records → read model → query/cache → UI/export → reconciliation/evidence. State tenant/entity/location/currency/time scope, owner, transition rules, approvals and recovery route. Include diagrams and distinguish observed imports from proposed or runtime-unverified links.

5. Domain and database correctness

Inspect TenantModuleConfiguration, TenantModuleGrant, TenantModuleConfigurationCommand, TenantModuleLifecycleOperation, TenantModuleLifecycleStep, TenantModuleLifecycleEvent. Validate identifiers, decimal/quantity/date precision, state transitions, foreign-tenant references, uniqueness, referential integrity, indexes, immutability, soft/hard deletion and data retention. Distinguish schema declarations, committed migration contents and constraints actually deployed. Reconcile economic/operational substance, not just balanced totals or a success status.

Domain-specific checks:

- Catalog availability is not released capability.
- Test direct services, jobs, APIs and routes under each entitlement mode.
- Separate new-work dependencies from obligations that must remain completable.

6. Connected workflows

Trace producer and consumer contracts with Identity, tenant access, settings and administration; Events, recovery, migrations and platform operations. Include accounting through the existing ACC/ACR roadmap without duplicating ledger ownership. Identify legitimate timing differences, material discrepancies, exception owner, replay/correction routes and effects of module suspension or permission changes. Record supported versus blocked variants.

7. Services, actions and reliability

Verify server-derived scope, validated inputs, central business-rule ownership, atomic writes, concurrency and safe error contracts. Prove duplicate, out-of-order and ambiguous-commit behavior. Check success responses that conceal incomplete downstream work, orphan effects, swallowed exceptions, durable conflict evidence, lease fencing, retry limits and dead letters. An event stored in an outbox is not proof of delivery or a downstream effect.

8. Access, control and privacy

Test direct actions/APIs independently of hidden buttons. Cover tenant/entity/location boundaries, role revocation, entitlements, fresh sensitive-action authorization, maker-checker, bulk operations and privileged overrides. Review public/employee/customer/supplier tokens, retention and redaction where relevant. Changes to authority must not strand mandatory settlement or leak historical data.

9. TanStack Query and rendered workflow

Trace DTOs, query keys/functions, server caches, mutations and consumers. Require every result-changing scope in identity and safe cancellation/removal on actor/tenant/entitlement change. Verify invalidation across connected domains and safe retry/optimistic updates. Distinguish loading, empty, denied, failed, stale, partial and true zero. Check amount/date/sign/total/page/export fidelity; keyboard, focus, mobile layout, EN/FR and costly-error prevention. Use authenticated browser evidence when available; otherwise label static review.

10. Reporting, performance and external scope

Reconcile authoritative source records with module balances, metrics and exports, preserving period/cumulative and operational/posted distinctions. Measure representative dataset sizes, latency, payload, queries/indexes and concurrency when an isolated environment is available; label unmeasured bottlenecks as hypotheses. Verify country/provider/version/effective-date and qualified approval for regulated claims, using current authoritative sources. Do not infer compliance from this template.

11. Verification protocol

Read package scripts and test bodies before execution. Existing focused starting point: services/modules/**tests**/module-admission.service.test.ts. Typical scoped command: npm test -- --runInBand --runTestsByPath followed by verified test paths. Run prisma validate/typecheck only as appropriate, and record limits. The repository build script invokes migration deployment; the close-assurance E2E composite deploys migrations and seeds. Do not run these as read-only checks. Use an explicitly disposable environment for DB-writing cases.

Record exact command, start/end time, candidate hash, environment, output location and passed/failed/skipped/blocked status. Preserve old logs. Test valid flow; denied no-read/no-write; duplicate same payload; conflicting payload; concurrent transition; rollback/retry; stale UI; scale/export; correction and recovery. State which were actually executed.

12. Findings and executable remediation

For each stable finding ID record classification, priority/severity, confidence, likelihood, impact, exact lines/symbols, test/runtime evidence, triggering scenario, expected/observed behavior, root cause, dependencies, financial/operational/privacy consequences, mitigations, correction owner, effort range and acceptance tests. Separate confirmed defects from plausible risk, unavailable capability and external blockers. Do not invent loss estimates or incidents.

Order work as immediate containment for demonstrated exposure → source/data/authority corrections → integration/recovery → query/performance/workflow improvements → historical repair investigation → release evidence. Reuse shared work and ACC/ACR packages; keep one owner per correction. Each task needs prerequisites, deliverable, measurable completion, repair/rollback implications and independently reviewed gate. Size implementation only after reproduction.

13. Reports and publication

Create a new immutable run under what-next/platform-module-audit/<date-run-id>/module-lifecycle-billing/ for logs, source hashes, screenshots and structured findings. Author final reports directly under docs/plan-of-action with module-lifecycle-billing-prefixed basenames to avoid collisions: audit-report.md, architecture-and-flow-map.md, remediation-plan.md and evidence-index.md, plus findings.json. A combined report is acceptable if it contains every section and clear evidence links. Preserve the previous package as historical.

Publish with node .agents/skills/docs-report-publisher/scripts/publish_reports.mjs plan-of-action. Every Markdown report must have a same-named nonempty PDF; preserve content and basename. Report the verified pair links.

14. Completion criteria

Every material capability and dependency has a recorded disposition; critical flows have traceable source-to-UI ownership; significant claims have evidence; failure paths are executed or explicitly unverified; strengths/defects/limitations/unknowns are distinct; the largest risks and the first concrete corrections are understandable to domain and engineering owners. No unrelated refactors, upgrades, speculative features, business-record mutation or deployment. Conclude within the evidence boundary.